

Installing Libraries

- i. pip install numpy
- ii. pip install numba

```
Anaconda Prompt
(base) C:\Users\rifaq>cd Desktop
(base) C:\Users\rifaq\Desktop>cd Lab_13
(base) C:\Users\rifaq\Desktop\Lab_13>pip install numpy
Requirement already satisfied: numpy in d:\miniconda3\lib\site-packages (2.4.4)
(base) C:\Users\rifaq\Desktop\Lab_13>pip install numba
Collecting numba
  Downloading numba-0.65.1-cp313-cp313-win_amd64.whl.metadata (3.0 kB)
Collecting llvmlite<0.48,>=0.47.0dev0 (from numba)
  Downloading llvmlite-0.47.0-cp313-cp313-win_amd64.whl.metadata (5.1 kB)
Requirement already satisfied: numpy>=1.22 in d:\miniconda3\lib\site-packages (from numba) (2.4.4)
Downloading numba-0.65.1-cp313-cp313-win_amd64.whl (2.8 MB)
  2.8/2.8 MB 2.0 MB/s 0:00:01
Downloading llvmlite-0.47.0-cp313-cp313-win_amd64.whl (38.1 MB)
  21.8/38.1 MB 106.6 kB/s eta 0:02:34
WARNING: Connection timed out while downloading.
WARNING: Attempting to resume incomplete download (21.8 MB/38.1 MB, attempt 1)
Resuming download llvmlite-0.47.0-cp313-cp313-win_amd64.whl (21.8 MB/38.1 MB)
  38.1/38.1 MB 676.7 kB/s 0:00:34
Installing collected packages: llvmlite, numba
Successfully installed llvmlite-0.47.0 numba-0.65.1
(base) C:\Users\rifaq\Desktop\Lab_13>
```

CUDA-compatible GPU was not available, therefore NumPy-based implementations were used for simulation and validation purposes.

Lab Tasks

Task 1: Element-wise Array Multiplication

- Write a CUDA program to perform element-wise multiplication of two arrays.
- Compare your output with NumPy's np.multiply().

```
task1_array_multiplic... 1
1 import numpy as np
2 import time
3
4 # input arrays
5 A = np.array([1, 2, 3, 4, 5], dtype=np.float32)
6 B = np.array([10, 20, 30, 40, 50], dtype=np.float32)
7
8 # start timer
9 start = time.time()
10
11 # element-wise multiplication
12 C = A * B
13
14 # end timer
15 end = time.time()
16
17 print("Array A:")
18 print(A)
19
20 print("\nArray B:")
21 print(B)
22
23 print("\nMultiplication Result:")
24 print(C)
```

```
25
26     print("\nExecution Time:", end - start)
27
28     # validation with NumPy
29     numpy_result = np.multiply(A, B)
30
31     print("\nNumPy Result:")
32     print(numpy_result)
33
34     if np.array_equal(C, numpy_result):
35         print("\nValidation Successful")
36     else:
37         print("\nValidation Failed")
```

Output

```
Anaconda Prompt
(base) C:\Users\rifaq>cd desktop
(base) C:\Users\rifaq\Desktop>cd Lab_13
(base) C:\Users\rifaq\Desktop\Lab_13>python task1_array_multiplication.py
Array A:
[1. 2. 3. 4. 5.]
Array B:
[10. 20. 30. 40. 50.]
Multiplication Result:
[ 10.  40.  90. 160. 250.]
Execution Time: 2.1219253540039062e-05
NumPy Result:
[ 10.  40.  90. 160. 250.]
Validation Successful
(base) C:\Users\rifaq\Desktop\Lab_13>
```

Task 2: Matrix Transpose

- Implement a CUDA kernel to compute the transpose of a given matrix.
- Validate with `np.transpose()`.

```
File Edit Selection View Go Run Terminal Help
Lab_13
EXPLORER
LAB_13
  check_cuda.py 1
  task1_array_multiplic... 1
  task2_transpose.py 1
  task3_row_sum.py 1
task2_transpose.py > ...
1 import numpy as np
2
3 # input matrix
4 A = np.array([
5     [1, 2, 3],
6     [4, 5, 6]
7 ], dtype=np.float32)
8
9 # transpose
10 transpose_matrix = A.T
11
12 print("Original Matrix:")
13 print(A)
14
15 print("\nTranspose Matrix:")
16 print(transpose_matrix)
17
```

```
18 # validation
19 numpy_transpose = np.transpose(A)
20
21 print("\nNumPy Transpose:")
22 print(numpy_transpose)
23
24 if np.array_equal(transpose_matrix, numpy_transpose):
25     print("\nValidation Successful")
26 else:
27     print("\nValidation Failed")
```

Output

```
(base) C:\Users\rifaq\Desktop\Lab_13>python task2_transpose.py
Original Matrix:
[[1. 2. 3.]
 [4. 5. 6.]]

Transpose Matrix:
[[1. 4.]
 [2. 5.]
 [3. 6.]]

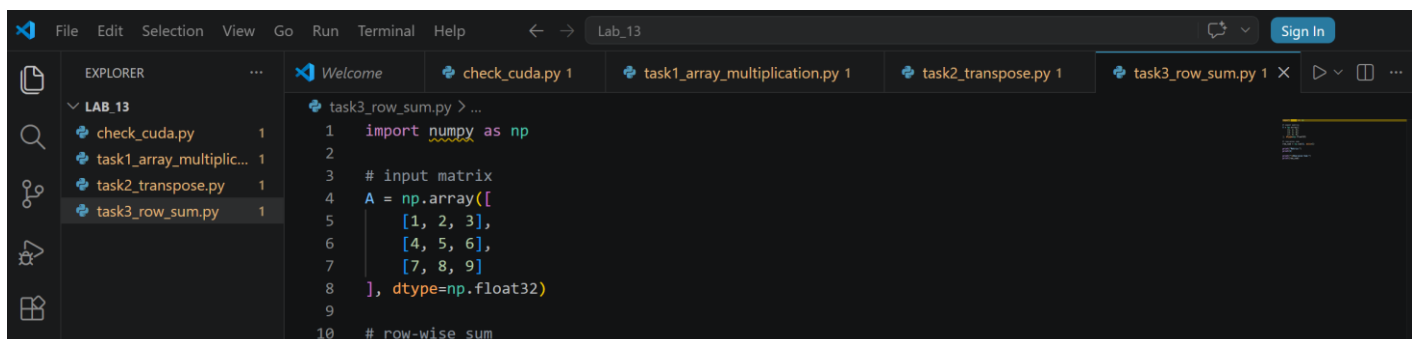
NumPy Transpose:
[[1. 4.]
 [2. 5.]
 [3. 6.]]

Validation Successful

(base) C:\Users\rifaq\Desktop\Lab_13>
```

Task 3: Row-wise Sum of Matrix

- Write a kernel to compute the sum of each row in a matrix using parallel threads.
- Output a 1D array of row sums.





```
13 print("Matrix:")
14 print(A)
15
16 print("\nRow-wise Sum:")
17 print(row_sum)
```

Output

```
(base) C:\Users\rifaq\Desktop\Lab_13>python task3_row_sum.py
Matrix:
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]

Row-wise Sum:
[ 6. 15. 24.]

(base) C:\Users\rifaq\Desktop\Lab_13>
```